

Estrutura e Processo da Fábrica Agêntica de Publicações (V4)

Este documento descreve detalhadamente o ecossistema de arquivos de instrução, portabilidade multi-IDE, e o fluxo operacional de ponta a ponta da **Fábrica Agêntica de Publicações (V4 Multi-formato)**.

1. Estrutura de Instruções e Portabilidade Multi-IDE

O ecossistema de instruções da Fábrica foi projetado para ser universal, resiliente e altamente eficiente no consumo de tokens, utilizando a técnica de **portabilidade sem duplicação de arquivos**.

1.1 CLAUDE.md (Fonte Única da Verdade)

O arquivo CLAUDE.md no raiz do projeto atua como o manual central de regras da fábrica. Ele contém as diretrizes de economia severa de tokens, definições de squads, mcp-servers, templates e fluxo de trabalho.

1.2 Mecanismo de Hardlinks

Para evitar a duplicação e desatualização de instruções entre diferentes IDEs e extensões agênticas, o projeto utiliza **links físicos (hardlinks)** no sistema de arquivos. Os seguintes caminhos apontam para o exato mesmo espaço de dados no disco que CLAUDE.md: - AGENTS.md (Padrão aberto consumido por diversas ferramentas agênticas) - .cursor/rules/fabrica-agentica.mdc (Cursor Project Rules) - .windsurfrules (Windsurf/Cascade) - .windsurf/rules/fabrica-agentica.md (Windsurf/Cascade) - .clinerules (Cline) - .github/copilot-instructions.md (GitHub Copilot)

1.3 Junctions de Pasta (agentic/)

As pastas de ferramentas e subagentes são espelhadas para fora do diretório oculto .claude/ usando **Junctions de Diretório** no Windows (ou links simbólicos em Linux/macOS): - agentic/skills → .claude/skills - agentic/agents → .claude/agents - agentic/commands → .claude/commands - agentic/mcp-servers → .claude/mcp-servers

1.4 Sincronização de MCP do VSCode

Como o VSCode utiliza um formato de configuração de MCP diferente das IDEs agênticas modernas (baseado em servers em vez de mcpServers), o arquivo .vscode/mcp.json é um arquivo traduzido de verdade. Ele é gerado automaticamente a partir do .mcp.json raiz através do script:

```
node scripts/sync-vscode-mcp.mjs
```

2. O Processo de Produção Passo a Passo (Esteira Editorial)

A Fábrica Agêntica opera através de **5 fases sequenciais e determinísticas** coordenadas por um Orquestrador Mestre.

Fase	Nome da Etapa	O que ela faz	Ferramentas e Recursos Utilizados
Fase 0	Preparação & Esboço	Captura dados do operador, cria estrutura de diretórios e gera a configuração de escopo.	Comando <code>/esbocar <tema>, scripts/parametros_obra.py, MCP db_state</code>
Fase 1	P&D (Inteligência Técnica)	Realiza varreduras web profundas e gera um dossiê técnico denso do assunto.	Skill pesquisador, subagente-pesquisador, scripts/indexar-dossie.py
Fase 2	Manufatura Tática	Redige os capítulos em paralelo divididos em lotes com tratamento de falhas.	Skills estrategista e redator-eita/redator-academico, scripts/pool-capitulos.py
Fase 2.5	Peer Review Autônomo	Audita o material escrito sob métricas objetivas (sintaxe, sobreposição, diagramas).	Skill revisor-tecnico, subagente-revisor-tecnico, scripts de auditoria/CI
Fase 3	Acabamento & Compilação	Une a obra, gera a ficha CIP, paleta cromática, capa 2D plana e compila para PDF/EPUB.	Skills compilador-abnt/tcc/artigo, templates Typst, compilar-para-pdf.py

3. Detalhamento de Ferramentas por Etapa

3.1 Fase 0: Preparação

- **Descrição:** O operador informa o tema central. O sistema analisa se deve derivar artigos ou e-books e cria a pasta de trabalho isolada em `output/<tipo>/<slug>/`.
- **Ferramentas:**
 - **Comando:** `/esbocar` ou `/produzir-obra-completa <slug>`
 - **MCP db_state:** Registra o estado inicial e rastreia o andamento.
 - **Script `scripts/parametros_obra.py`:** Define os parâmetros da obra, regexes de citação e escalas de tamanho (P/M/G).

3.2 Fase 1: Pesquisa e Desenvolvimento

- **Descrição:** Varre a web e documentações técnicas em busca de dados relevantes de alta qualidade para consolidar um dossiê estruturado.
- **Ferramentas:**
 - **Skill pesquisador & subagente-pesquisador:** Realizam as varreduras via comandos nativos WebSearch e WebFetch.
 - **Script scripts/indexar-dossie.py:** Constrói um índice RAG local (TF-IDF puro) do dossiê gerado, permitindo consultas focadas por relevância sem inflar o contexto dos agentes.

3.3 Fase 2: Manufatura Paralela em Lotes

- **Descrição:** Escreve os capítulos de forma modular aplicando padrões estruturados (EITA para livros comerciais, ACAD para produções científicas/acadêmicas).
- **Ferramentas:**
 - **Script scripts/pool-capitulos.py:** Orquestra a execução concorrente. Limita o envio de capítulos a lotes de 4 de cada vez (evitando bloqueios de TPM/RPM da API) e executa retenção automática com backoff exponencial (máx 3 tentativas).
 - **Subagentes de Manufatura:** subagente-redator-capitulo (livros) ou subagente-redator-secao-tcc (TCC).
 - **Skills de Redação:** estrategista (pilares didáticos), redator-eita (estilo comercial), redator-academico (tom impessoal, citação ABNT autor-data), redator-ebook (adaptação de tom leve).
 - **Template:** templates/template_eita.md (garante estrutura pedagógica de 7 seções, código técnico e diagramas).

3.4 Fase 2.5: Peer Review Autônomo

- **Descrição:** Um processo rigoroso de teste e validação baseado em scripts determinísticos de análise e não em impressões subjetivas dos modelos.
- **Ferramentas:**
 - **Script scripts/auditar-obra.py:** Detecta sobreposições de capítulos, grafias inconsistentes de termos técnicos e trechos truncados.
 - **Script scripts/validar-codigo.py (CI de Código):** Valida a sintaxe de todos os blocos de código existentes na obra de forma estática (Python, JS, TS, Bash, etc.).
 - **Script scripts/renderizar-diagramas.py:** Renderiza todos os blocos ``mermaid para imagens PNG com cache de hash e validação de sintaxe.
 - **Skill revisor-tecnico & subagente-revisor-tecnico:** Lê os JSONs gerados pelos scripts de auditoria e corrige pontualmente os trechos apontados como reprovados.

3.5 Fase 3: Acabamento & Exportação Editorial

- **Descrição:** Reúne as peças textuais e gráficas, gera elementos pré e pós-textuais normalizados e compila para os formatos de distribuição final.
- **Ferramentas:**
 - **Skills Compiladoras:** compilador-abnt (Livros), compilador-tcc (TCC), compilador-artigo (Artigo Científico).

- **Subagente subagente-ilustrador:** Gera ilustrações 2D flat para capítulos via HTML/CSS e Playwright sem custo de API.
 - **Scripts Editoriais:**
 - `scripts/metadados_livro.py`: Deriva paleta cromática, gera ficha catalográfica CIP e sinopse da contracapa.
 - `scripts/gerar-cap-a-ebook-padrao.py`: Renderiza a capa 2D matte plana profissional oficial em PNG.
 - `scripts/validar-abnt-tcc.py`: Valida o formato pré-textual acadêmico.
 - `scripts/gerar-epub.py`: Converte e-books derivados para EPUB reflowable nativo.
 - **Mecanismos de PDF:** `compile-para-pdf.py` ou `scripts/convert-md-pdf.ps1` executam a conversão nativa de **Pandoc** → **.typ** → **Typst** (altamente profissional, ultra veloz, margens ABNT, Times New Roman 12pt, sumário automático e paginação).
 - **MCP pdf_gen:** Fornece um fallback opcional utilizando CloudConvert caso a stack local falhe.
-

4. Diretrizes Globais de Eficiência (Token Economy)

Para garantir sustentabilidade operacional em sessões longas, a Fábrica utiliza 4 ganchos (*hooks*) de automação de contexto baseados no framework *Lean-CTX*:

1. **lean-ctx (Leitura Cirúrgica):** Realiza `grep_search` focado para ler apenas trechos exatos de arquivos grandes, em vez de ler arquivos por inteiro.
2. **headroom (Compressão de Logs):** Trunca logs de terminal com mais de 7 linhas, mantendo apenas 3 linhas superiores e 4 inferiores para o diagnóstico de erros.
3. **caveman (Pensamento e Chat Compactos):** Mantém a comunicação interna em formato telegráfico PT-BR, livre de conversas fiadas e saudações desnecessárias.
4. **rtk-memory (Scratchpad Dinâmico):** Grava aprendizados e erros de compilação em tempo de execução no bloco `RTK SCRATCHPAD` no fim do `CLAUDE.md`.
5. **pre-flight-check:** Executa testes de sintaxe e build antes de realizar commits no repositório.